Compiler for Processing in Memory (PIM)

End-to-End Architectural Specification, Intermediate Representation, Instruction Mapping, Analytical Cost Modeling, and Performance Evaluation

Complete Comprehensive Project Report — All Lifecycle Phases & Verified Implementation

Project Domain: Advanced Compiler Construction & Heterogeneous PIM Architecture	**Implementation Status:** Phase 1, Phase 2, Phase 4 Complete (Phase 3 Decoupled)
Target Hardware: Dual-Domain: Host CPU (Von Neumann) + Near-DRAM Row-Buffer ALUs	**Automated Test Suite:** 73 Backend Tests (100% Pass) | Frontend Build Verified
Technology Stack: Python 3.12 (FastAPI, 3AC Engine, Analytical Model), React 19, Vite	**Visual Analytics:** Dual Multi-View: Stacked Bar Chart & SVG Dynamic Donut/Pie

1. Executive Summary & Architectural Motivation

1.1 The Von Neumann Memory Wall: In classical von Neumann architecture, computational logic (CPU/ALU) is physically separated from main memory (DRAM) by an external printed circuit board (PCB) bus. While microprocessor computation speeds have scaled exponentially over decades, memory bus latency and bandwidth have improved at merely ~7% annually. For data-intensive workloads (vector reductions, matrix operations, and large array summations), modern systems spend **60% to 80% of total system energy and execution cycles purely transporting data** across high-capacitance off-chip memory buses rather than performing arithmetic computation.

1.2 The Processing in Memory (PIM) Paradigm: PIM fundamentally reverses this model by moving computational logic directly into or immediately adjacent to the DRAM substrate. By placing compact, energy-efficient Arithmetic Logic Units (ALUs) directly at the DRAM bank row buffers (Near-Memory Computing), operands are accessed across thousands of internal bitlines with **terabytes-per-second internal bandwidth** and sub-nanosecond latencies, bypassing the external system bus entirely.

1.3 Purpose and Objectives of this Project: This project designs, implements, and evaluates an end-to-end specialized compiler and analytical performance evaluation framework for Processing in Memory. The system parses high-level C programs, synthesizes an AST, enforces scoped semantic checks, lowers code into Three-Address Code (3AC) Intermediate Representation, maps data-parallel instructions to PIM or host CPU fallback, and calculates exact analytical execution costs across three strategies using a calibrated four-term cost model.

Core Academic Contribution: The compiler does not offload instructions blindly; it features a threshold-aware cost engine that quantifies trade-offs. If bus transfer savings do not exceed PIM kernel launch and synchronization overheads, the compiler automatically retains CPU execution, guaranteeing mathematically optimal scheduling.

2. End-to-End System Pipeline & Architecture

Stage	Module Name	Input Data	Output Data	Core Transformation & Responsibilities
Stage 1	Lexical Analyzer scanner.py	Raw C Source Code	Token Stream (5 categories)	Regex-based tokenization into keywords, identifiers, operators, delimiters, and numeric literals with coordinate tracking.
Stage 2	Syntax Parser parser.py	Token Stream	Abstract Syntax Tree	Recursive-descent syntactic analysis enforcing concrete grammar rules for variables, arrays, expressions, and loops.
Stage 3	Semantic Analyzer semantic.py	AST Nodes	Decorated AST & Symbol Table	Scope validation, type checking, static array bounds checking, and constant division-by-zero detection.
Stage 4	IR Generator ir_generator.py	Decorated AST	3AC Quadruples	Lowers control flow and compound expressions into sequential 4-tuple Three-Address Code with explicit temporaries and jump labels.
Stage 5	PIM Mapper & Cost pim_mapper.py cost_model.py	3AC Quadruples	Mapped IR & 4-Term Strategy Costs	Classifies ALU ops to DRAM_ROW_BUFFER vs HOST_CACHE, enforces CPU fallback, and calculates exact cycle costs for CPU_ONLY, DIRECT_PIM, and TRANSFORMED_PIM.

3. Phase 1: Compiler Frontend Design & Implementation

3.1 Lexical Analysis (Scanner):

The lexical scanner (scanner.py) processes raw C source code and decomposes it into a continuous stream of strongly classified lexical tokens. It eliminates whitespace and both single-line (//) and multi-line (/* ... */) comments while recording exact 1-indexed line and column coordinates for every token to provide human-readable diagnostic error reporting.

Tokens are strictly partitioned into five distinct academic categories:

- **Keywords:** Reserved language constructs including int, float, for, while, if, else, return, void.
- **Identifiers:** Programmer-defined variables, arrays, and function names matching regex [a-zA-Z_][a-zA-Z0-9_]*.
- **Operators:** Arithmetic (+, -, *, /, %), Relational (==, !=, <, <=, >, >=), Bitwise (&, |, ^), and Assignment (=).
- **Delimiters:** Syntactic punctuation tokens including (,), {, }, [,], ;, and ,.
- **Numeric Literals:** Integer constants (e.g., 0, 42, 100) validated as positive numerical integers.

3.2 Syntax Analysis & AST Grammar (Parser):

The syntax parser (parser.py) implements a recursive-descent parsing technique with predictive single-token lookahead. It validates the syntactic correctness of the token stream against the supported C language grammar and builds a hierarchical Abstract Syntax Tree (AST). Operator precedence is strictly enforced through layered recursive routines: relational operators bind looser than additive operators (+, -), which in turn bind looser than multiplicative operators (*, /, %).

AST Node Class	Structural Fields & Attributes	Grammar Role & Representation
`Program`	declarations: List[ASTNode]	Root node containing global variables, arrays, and function definitions.
`FunctionDecl`	name, return_type, params, body	Function header declaration and compound statement body (e.g., main()).
`VarDecl / ArrayDecl`	name, type, size, initializer	Scalar variable and fixed-size static array declarations with initializer lists.
`ForStmt / WhileStmt`	init, condition, update, body	Iteration loops defining iteration count, induction step, and loop body statements.
`IfStmt`	condition, then_branch, else_branch	Conditional selection branching with optional else clause.
`AssignStmt`	target, value	Assignment to scalar variable or indexed array element (target[index] = expr).
`BinaryExpr`	op, left, right	Binary arithmetic, logical, or comparison expression node.
`ArrayAccess`	array_name, index_expr	Indexed read access to array memory element (e.g., arr[i]).

3.3 Scoped Semantic Analysis & Symbol Table:

The semantic analyzer (semantic.py) conducts static semantic checks over the AST before code generation. It manages a hierarchically chained SymbolTable with parent pointer references to support lexical block scoping. Static checks enforced include: (1) detection of undeclared identifiers, (2) rejection of duplicate declarations within the same scope, (3) prevention of scalar indexing as arrays, (4) array initializer size validation against declared capacity, (5) compile-time static array bounds checking for constant index expressions, and (6) constant division-by-zero detection.

4. Phase 2: Intermediate Representation (Three-Address Code)

4.1 Three-Address Code (3AC) Quadruple Specification: The IR generator (ir_generator.py) lowers the high-level AST into a linear sequence of Three-Address Code Quadruples. Each instruction conforms strictly to a 4-tuple schema: (opcode, arg1, arg2, result). Compound nested expressions are flattened into sequential atomic instructions using compiler-generated temporary variables (t0, t1, t2, ...), while iteration loops and conditional branches are lowered into explicit comparison operations, conditional jumps (JUMP_IF_FALSE), unconditional jumps (JUMP), and target labels (LABEL).

Opcode Family	3AC Instructions	Quadruple Structure	Operational Semantics
Arithmetic	`ADD, SUB, MUL, DIV, MOD`	`(OP, src1, src2, dest)`	Computes dest = src1 OP src2.
Bitwise	`BITAND, BITOR, BITXOR`	`(OP, src1, src2, dest)`	Bitwise logical operations on integer registers.
Relational	`LT, GT, LE, GE, EQ, NE`	`(OP, src1, src2, dest)`	Boolean comparison returning 1 (true) or 0 (false) to dest.
Memory / Array	`LOAD_INDEX, STORE_INDEX`	`(LOAD_INDEX, arr, idx, dest)` `(STORE_INDEX, val, idx, arr)`	Array element read/write with 4-byte scaled offset arithmetic.
Control Flow	`LABEL, JUMP, JUMP_IF_FALSE`	`(LABEL, None, None, L1)` `(JUMP_IF_FALSE, cond, None, L1)`	Defines jump target or evaluates condition to branch.

4.2 Array Address Arithmetic: Array accesses are lower-level operations that require calculating physical byte offsets. For a 32-bit integer array access arr[i], the IR generator emits: (MUL, i, 4, t0) followed by (LOAD_INDEX, arr, t0, t1), ensuring accurate physical memory modeling.

5. Phase 2: PIM Hardware Instruction Mapping & Fallback

5.1 The Near-Memory Execution Model: The PIM instruction mapper (pim_mapper.py) inspects the generated 3AC stream and partitions each instruction between the Host CPU and the near-memory PIM ALUs. The architectural model assumes simple SIMD/vector ALUs embedded in the DRAM bank periphery directly beside the row buffer. When an operation accesses array data located in the DRAM bank, the instruction can execute in-situ on the row buffer data, avoiding the memory bus traversal.

5.2 Instruction Classification & ISA Partitioning Table:

IR Opcode	Target Hardware	Mapped Target Opcode	Target Data Region	Architectural & Mapping Rationale
`ADD`	**PIM (Near-Memory)**	`PIM_ADD`	DRAM_ROW_BUFFER	Supported in-situ integer vector/scalar addition in row buffer.
`SUB`	**PIM (Near-Memory)**	`PIM_SUB`	DRAM_ROW_BUFFER	Supported in-situ integer subtraction in row buffer.
`MUL`	**PIM (Near-Memory)**	`PIM_MUL`	DRAM_ROW_BUFFER	Supported near-memory multiplication logic unit.
`BITAND/OR/XOR`	**PIM (Near-Memory)**	`PIM_BITAND / ...`	DRAM_ROW_BUFFER	Bitwise logical bitline operations directly inside DRAM.
`DIV / MOD`	**Host CPU (Fallback)**	`CPU_DIV / CPU_MOD`	HOST_CACHE	Hardware division requires multi-cycle iterative dividers unavailable in DRAM ALUs. CPU fallback.
`JUMP / JUMP_IF_FALSE`	**Host CPU (Fallback)**	`CPU_JUMP / ...`	HOST_CACHE	Program control flow, branch prediction, and instruction sequencing remain governed by host CPU.
`RETURN`	**Host CPU (Fallback)**	`CPU_RETURN`	HOST_CACHE	Function termination and stack unwind handled by host CPU runtime.

5.3 CPU Fallback Mechanism: PIM DRAM chips possess strict silicon area and thermal budgets that prevent embedding complex floating-point units, hardware divider circuits, or speculative branch predictors. The compiler enforces transparent CPU fallback: whenever an unsupported opcode (such as DIV or MOD) or scalar control flow instruction is encountered, the instruction target is assigned to CPU with HOST_CACHE data region. The compiler inserts necessary bus synchronization boundaries to safely pass intermediate results between DRAM row buffers and the host CPU.

6. Architectural Decoupling of Phase 3 Simulation Engine

6.1 The Dual-Approach Divergence Problem: In early compiler explorations, a separate software cycle simulator (Phase 3) was prototyped to trace cycle-by-cycle execution of CPU and PIM instructions on synthetic memory traces. However, maintaining a standalone software interpreter alongside a formal compiler optimization pipeline created severe engineering and pedagogical challenges: (1) Divergence between interpreter execution traces and static compiler analysis, (2) Massive execution overhead for large array benchmarks, and (3) Redundant, competing definitions of cycle costs between the simulator and the compiler cost model.

6.2 The Compiler-Centric Analytical Solution: In industrial and academic compiler engineering (e.g., LLVM, GCC, and high-performance heterogeneous compilers like TVM), scheduling and offloading decisions are governed by a **static analytical cost model** rather than cycle-by-cycle interpreted software emulation. Consequently, Phase 3 software simulation was deliberately decoupled and stripped from the primary compiler execution path. The compiler now relies on a unified, mathematically rigorous analytical cost model that acts as the single source of truth.

Architectural Rationale: Decoupling Phase 3 execution simulation streamlined the project into a cohesive 5-stage compilation and evaluation framework. It eliminated architectural discrepancies, accelerated compiler evaluation to sub-millisecond response times, and enabled interactive real-time sensitivity analysis across hardware parameter spaces.

7. Phase 4: Analytical Cost Model & Performance Evaluation

7.1 The Four-Term Cost Formulation: The compiler evaluates candidate execution strategies using a calibrated analytical cost model (cost_model.py). For any strategy *S*, the total execution cost (in clock cycles) is defined by the exact equation:

Total Cost (cycles) = Compute Cost + Data Transfer Cost + Layout Cost + Synchronization Overhead

- **Compute Cost:** Cycles spent executing ALU math operations on either CPU or PIM units.
- **Data Transfer Cost:** Cycles consumed moving operand arrays over the physical memory bus: Bytes Transferred / Bus Bandwidth.
- **Layout Cost:** Incurred when memory layout reorganization is necessary for contiguous PIM vectorization.
- **Synchronization Overhead:** Host-to-PIM kernel dispatch latency, barrier synchronization, and cache flush penalties.

7.2 Hardware Model Parameters & Baseline Assumptions:

Parameter	Symbol / Key	Baseline Value	Architectural Basis & Description
CPU Op Compute Latency	`cpu_cycles_per_op`	1.0 cycle	Pipelined superscalar host CPU execution latency per basic integer ALU op.
PIM Op Compute Latency	`pim_cycles_per_op`	1.2 cycles	Slightly slower clock domain of in-DRAM ALUs due to low-power silicon process.
Memory Bus Bandwidth	`memory_bandwidth`	2.5 Bytes/cycle	Effective data transfer throughput over the off-chip host memory interconnect.
PIM Launch / Sync Overhead	`pim_sync_overhead`	60.0 cycles	Fixed latency for command dispatch, TLB/memory fence, and barrier synchronization.
Layout Transform Cost	`layout_transform_cost`	4.0 cycles/elem	Overhead of transposing or striding array elements for aligned SIMD vector execution.

7.3 Execution Strategies Compared:

- **CPU_ONLY:** All operations execute on the host CPU. All array operands must traverse the off-chip memory bus into CPU L1/L2 caches. Data transfer cost is high; synchronization overhead is zero. Serves as baseline with speedup = 1.00x.
- **DIRECT_PIM:** Eligible vector/array operations execute in-situ in the DRAM row buffer. Memory bus transfer is reduced to scalar results or boundary conditions; pays the 60-cycle PIM launch/sync overhead.
- **TRANSFORMED_PIM:** Evaluated when array memory layout requires transposition or alignment before offloading. Feasible only when memory layout transformation can be resolved statically.

7.4 Strict Floating-Point Arithmetic Invariants:

A critical visual and mathematical defect in educational dashboards is rounding mismatch (e.g., displaying rounded integers 14 + 4 + 60 = 78.4). The project guarantees strict floating-point precision formatted to exactly **1 decimal place** across all UI components, tables, and chart segments. For example, in the array sum benchmark (N = 4):

14.4 (Compute) + 4.0 (Transfer) + 60.0 (Sync) = 78.4 Total Cycles (Identity rigorously preserved).

7.5 Data Movement & Memory Traffic Distinction:

The compiler strictly delineates three distinct memory metrics: (1) **Declared Memory Footprint:** Static allocation capacity (N * sizeof(int)), (2) **Bytes Accessed by Program:** Dynamic memory reads and writes performed during execution, and (3) **Modeled Bus Traffic:** Actual bytes transferred across the physical off-chip host bus. In PIM execution, Bytes Accessed remains identical, but Modeled Bus Traffic drops toward zero for local row buffer operations.

8. User Interface, Interactive Visualizations & Sandbox

8.1 Unified PIM Mapping & Performance Analysis Dashboard: Rather than scattering data across isolated tabs, Phase 2 and Phase 4 are unified into a cohesive, professional 4-card dashboard layout:

- **Card 1 (Three-Address Code IR):** Monospace display of generated 3AC quadruples with line numbers, opcodes, arguments, and result destinations.
- **Card 2 (Instruction Mapping Table):** Comprehensive table mapping each 3AC instruction to its assigned hardware target (CPU vs PIM), target memory region (HOST_CACHE vs DRAM_ROW_BUFFER), and architectural fallback justification.
- **Card 3 (Strategy Comparison Cards):** Side-by-side strategy cards displaying Total Estimated Cycles, Speedup Badge relative to CPU_ONLY (1.00x baseline), and Feasibility status.
- **Card 4 (Cost Breakdown Table & Multi-View Visualizations):** Detailed breakdown table showing Compute, Transfer, Layout, and Sync costs alongside multi-view charts.

8.2 Multi-View Visual Analytics (Bar & Interactive SVG Donut/Pie):

Card 4 incorporates a professional 3-way segmented view switcher ([All Views], [Bar], [Pie]):

- **Stacked Bar Chart:** Renders cost components as stacked horizontal bars. Crucially, non-zero segments enforce a minimum width threshold (min-width: 28px) ensuring that smaller components (e.g., 4.0 cycles transfer) remain clearly legible with high-contrast text.
- **Interactive SVG Donut / Pie Chart:** Custom-built SVG chart utilizing mathematically exact trigonometric arc paths (d='M ... A ...'). Features a dynamic center readout that displays the total strategy cycles by default and updates dynamically on slice hover to show the specific component cycles, percentage share, and component label.

8.3 Interactive Cost Model Assumptions Sandbox:

A collapsible configuration drawer enables users to test hypothetical hardware architectures in real time. Users can adjust CPU latency, PIM latency, memory bus bandwidth, PIM synchronization overhead, and layout penalty. All strategy costs, speedup factors, stacked bar segments, and SVG pie charts recalculate instantly on the client while preserving the backend as the single source of truth.

9. Verification, Test Suite & Benchmark Evaluation

9.1 Automated Test Suite (73 Tests Passing — 100% Success Rate): The entire compiler pipeline, cost model, instruction mapper, and REST API are verified by a comprehensive pytest test suite comprising 73 automated tests.

Test Suite Module	Test Count	Primary Verification Scope & Edge Cases Covered	Status
`test_lexer.py`	8 tests	5-way token categorization, comment stripping, coordinate tracking, invalid characters.	**100% PASS**
`test_parser.py`	9 tests	Operator precedence, static array declarations, for/while loops, syntax errors.	**100% PASS**
`test_semantic.py`	7 tests	Scope analysis, undeclared variables, static array bounds check, constant division by zero.	**100% PASS**
`test_ir_generator.py`	7 tests	3AC Quadruple structure, temporary generation, loop lowering, array indexing math.	**100% PASS**
`test_pim_mapper.py`	8 tests	PIM ALU op mapping, CPU fallback for DIV/MOD/jumps, data region assignment.	**100% PASS**
`test_cost_model.py`	8 tests	4-term exact cost equation, small vs large workload selection, fallback cost tracking.	**100% PASS**
`test_performance.py`	14 tests	Strategy comparisons, speedup math, zero denominator safety, sensitivity analysis.	**100% PASS**
`test_compiler_orchestration.py`	5 tests	End-to-end pipeline execution, error cascading, demonstration program verification.	**100% PASS**
`test_api.py`	7 tests	FastAPI REST endpoints: /api/compile, /api/evaluate, health checks, error responses.	**100% PASS**
TOTAL	**73 Tests**	**Comprehensive Coverage Across All Phases & Error Modalities**	**73 / 73 PASS**

9.2 Benchmark Evaluation: Workload Scale Comparison:

To evaluate compiler behavior across workload scales, two canonical benchmarks were analyzed: (1) **Small Array Sum (N = 4):** Demonstrates threshold awareness. With only 4 elements, CPU_ONLY requires 28.0 cycles (12.0 compute + 16.0 bus transfer), whereas DIRECT_PIM requires 78.4 cycles (14.4 compute + 4.0 transfer + 60.0 sync). The compiler correctly selects **CPU_ONLY** as optimal, proving that offloading overhead is not blindly incurred.
(2) **Large Vector Reduction (N = 10,000):** CPU_ONLY incurs ~70,000 cycles due to massive memory bus traffic. DIRECT_PIM eliminates off-chip data movement, reducing total execution cost to ~12,100 cycles, yielding a **~5.78x speedup**.

10. Academic Viva & Technical Defense Guide

Q1: Why is PIM not always faster than CPU execution for small arrays?

Answer: PIM kernels incur an unavoidable host-to-memory dispatch and synchronization latency (modeled at 60 cycles). For small workloads (N < 30), the memory bus transfer savings (e.g., saving 12 bytes = ~5 cycles) are far smaller than the 60-cycle sync penalty. Our compiler detects this crossover point analytically and preserves CPU execution.

Q2: Why can't all C operations (like division or modulo) execute in PIM?

Answer: DRAM row-buffer ALUs have severe silicon area and power dissipation limits. Iterative dividers or floating-point units require complex logic trees and generate excessive heat that threatens DRAM cell charge retention. Our compiler transparently routes DIV, MOD, and branches to host CPU fallback.

Q3: Why did you decouple Phase 3 software simulation in favor of the analytical cost model?

Answer: Production compilers (like LLVM and TVM) schedule code using analytical cost models, not runtime emulation. A standalone software simulator introduced duplicate maintenance, slow execution on large datasets, and divergent cycle accounting. Decoupling Phase 3 created a unified, deterministic, sub-millisecond evaluation pipeline.

Q4: What is the difference between Declared Memory Footprint and Modeled Bus Traffic?

Answer: Declared Footprint is the static allocation size in RAM. Bus Traffic is the actual volume of bytes that must cross the physical CPU-DRAM bus. PIM accesses DRAM data locally within the row buffer at TB/s internal bandwidth, reducing modeled external bus traffic to near zero.

11. Conclusion & Future Research Extensions

Project Conclusion: This project successfully delivers a fully verified, end-to-end compiler and analytical performance evaluation framework for Processing in Memory. By integrating lexical, syntactic, and semantic frontend analysis with Three-Address Code intermediate representation, intelligent PIM instruction mapping, automatic CPU fallback, and a 4-term analytical cost model with multi-view visualizations, the system bridges the gap between high-level software abstractions and near-memory computer architecture.

Future Research Directions:
• **Multi-Bank DRAM Parallelism:** Extending the IR lowerer to partition vector operations across multiple concurrent DRAM banks.
• **LLVM Backend Integration:** Lowering 3AC quadruples into LLVM IR to leverage existing production vectorizer and register allocation passes.
• **FPGA Hardware Emulation:** Synthesizing near-bank ALUs on an FPGA-based RISC-V platform for cycle-accurate physical validation.

Project Verification Signoff:	**Academic Defense Review Status:**
Compiler Architecture, IR Lowering & Cost Model: **100% COMPLETE** All 73 Unit & Integration Tests Passing (pytest) Frontend Reactive Dashboard & Visualizations: **VERIFIED**	Capstone Review Committee Assessment: **APPROVED FOR VIVA** Department of Computer Science & Engineering Academic Year 2025–2026